Daily Maintenance TSG

Tags	no-headers, report_subprocess_monitoring
Team Owner	Proc and Roll
Status	ACTIVE
Last Audit	Dec 6, 2022

- Data Maintenance tasks
  - Data Maintenance JIRA Filter
- Diagnosis and Mitigation
  - Support KBA
  - Checking Status
  - Checking prune expired data task
  - Disable maintenance tasks
- Escalating to Engineering
  - Logs
  - Database Queries
  - Maintenance Configuration
- Post Mortem Articles - 🛠️ WIP
  - Daily Maintenance Pruner Limits
  - Daily Maintenance proper way to increase Raw Retention Data Threshold
  - Daily Maintenance failing to complete because of config app pruner
    - MySQL client ran out of memory
    - MySQL server has gone away

Data Maintenance tasks

Daily Maintenance executes the following tasks:

```
    if model in (SILO_MODEL_CU, SILO_MODEL_MCU):
        tasks.extend(
            [
                maint_tasks.CollectorDBCleanup(dbc, logger, settings),
                maint_tasks.PruneExpiredCollectedData(dbc, logger, settings, debug=_debug),
                maint_tasks.PurgeOldTempTables(dbc, logger, settings),
            ]
        )
    elif model in (SILO_MODEL_DB, SILO_MODEL_AIO):
```

```
        tasks.extend(
            [
                maint_tasks.ServiceAndWarrantyExpiration(dbc, logger, settings),
                maint_tasks.SSLCertificateExpiration(dbc, logger, settings),
                maint_tasks.ProcessExecutiveReportData(dbc, logger, settings),
                maint_tasks.DynamicAppTableMaint(dbc, logger, settings),
                maint_tasks.PruneExpiredCollectedData(dbc, logger, settings, debug=_debug),
                maint_tasks.AuditOverlappingDeviceIPAddresses(dbc, logger, settings),
                maint_tasks.DropUnmappedPresentationColumnsInJournalDA(dbc, logger, settings),
                maint_tasks.FixUnclassifiedIPs(dbc, logger, settings),
                maint_tasks.CleanupTmpDirs(dbc, logger, settings),
                maint_tasks.OrphanedDeviceIdentities(dbc, logger, settings),
                maint_tasks.CleanupSchedulerVEvent(dbc, logger, settings),
                maint_tasks.ReportFilesCleanup(dbc, logger, settings),
                maint_tasks.DiscoveryLogPersistCleanup(dbc, logger, settings),
                maint_tasks.RemoveStaleDeviceHardware(dbc, logger, settings),
                maint_tasks.DisableInactiveInterfaces(dbc, logger, settings),
                maint_tasks.OrphanedCustomAttributes(dbc, logger, settings),
                maint_tasks.DCMROldLinks(dbc, logger, settings),
                maint_tasks.PruneOrphanInterfaces(dbc, logger, settings),
            ]
        )
    elif model is SILO_MODEL_AP:
        tasks.extend(
            [
                maint_tasks.ServiceAndWarrantyExpiration(dbc, logger, settings),
                maint_tasks.CleanupTmpDirs(dbc, logger, settings),
            ]
        )

    if model in (SILO_MODEL_CU, SILO_MODEL_MCU, SILO_MODEL_AIO):
        tasks.extend(
            [
                maint_tasks.PruneCollectorDynamicAppCache(dbc, logger, settings),
            ]
        )
        tasks.extend(
            [
                maint_tasks.PruneCollectorMiscCache(dbc, logger, settings),
            ]
        )
        tasks.extend(
            [
                maint_tasks.PruneCollectorSnmpCache(dbc, logger, settings),
            ]
        )
        tasks.extend(
            [
                maint_tasks.CleanCollectorProcessesTable(dbc, logger, settings),
            ]
        )
```

1. **CollectorDBCleanup** - Performs comprehensive database cleanup operations on collector databases.

2. **PruneExpiredCollectedData** - Removes performance data that has exceeded its retention period.
3. **PurgeOldTempTables** - Drops temporary database tables that are no longer in use.
4. **ServiceAndWarrantyExpiration** - Checks and updates service and warranty expiration information for devices.
5. **SSLCertificateExpiration** - Monitors SSL certificates and flags those approaching expiration.
6. **ProcessExecutiveReportData** - Aggregates and processes data needed for executive-level reporting.
7. **DynamicAppTableMaint** - Optimizes and repairs Dynamic Application database tables.
8. **AuditOverlappingDeviceIPAddresses** - Identifies and resolves cases where multiple devices share IP addresses.
9. **DropUnmappedPresentationColumnsInJournalDA** - Removes presentation columns that no longer map to data in journal Dynamic Applications.
10. **FixUnclassifiedIPs** - Attempts to classify IP addresses that haven't been properly categorized.
11. **CleanupTmpDirs** - Removes old files from temporary directories to free disk space.
12. **OrphanedDeviceIdentities** - Identifies and cleans up device identities that no longer have associated devices.
13. **CleanupSchedulerVEvent** - Removes old scheduler event data to prevent database bloat.
14. **ReportFilesCleanup** - Deletes old report files based on configured retention policies.
15. **DiscoveryLogPersistCleanup** - Removes old discovery logs that have been persisted beyond their retention period.
16. **RemoveStaleDeviceHardware** - Removes hardware information for devices that are stale or have been removed.
17. **DisableInactiveInterfaces** - Automatically disables network interfaces that have shown no traffic for a configured period.
18. **OrphanedCustomAttributes** - Identifies and removes custom attributes that no longer have associated entities.
19. **DCMROldLinks** - Cleans up old or broken links in the Device Component Mapping Repository.
20. **PruneOrphanInterfaces** - Removes network interfaces that no longer have associated devices.
21. **PruneCollectorDynamicAppCache** - Clears outdated entries from the Dynamic Application cache on collectors.

22. **PruneCollectorMiscCache** - Removes expired entries from miscellaneous caches on collectors.
23. **PruneCollectorSnmpCache** - Cleans up the SNMP data cache on collectors to improve performance.
24. **CleanCollectorProcessesTable** - Removes old entries from the collector processes tracking table.

Data Maintenance JIRA Filter

[Escalations and Support Defects]

Diagnosis and Mitigation

Support KBA

The support KBA contains information on how to diagnose Daily Maintenance when the process fails and steps for mitigating failure. This should be the first document consulted if the process is failing to complete:

ScienceLogic Support

Checking Status

Daily Maintenance start and end times are logged in the database. To see the start and end times of the most recent executions of the process, a query like the following may be useful:

```
SELECT start_time, end_time FROM master_logs.system_runs
WHERE sp_id = 4 and mod_id = <module_id> ORDER BY start_time DESC LIMIT 10;
```

`mod_id` should be the ID of the appliance of interest.

Checking prune expired data task

One of the many tasks that Daily Maintenance runs includes the pruning of expired data. This task has been known to cause performance issues in the past and there have been many improvements to it as a result. It is often the task that consumes the most resources for Daily Maintenance.

The following query can be used to check the progress of the prune expired data task, which shows which expired data pruners have been completed:

```
SELECT p_id, date_start, date_end, elapsed
FROM master_logs.pruner_log
ORDER BY date_start desc;
```

Information about a particular pruner can be gathered from `master.system_settings_retention` by querying on the pruner's `ret_id`,

which matches the `p_id` from the previous query, or by checking out the [init_data file on bitbucket](bitbucket).

For example:

```
SELECT ssr.name, ssr.db, ssr.tbl, ssr.date_field, ssr.retention_default
FROM master.system_settings_retention ssr
WHERE ssr.ret_id =59;

+--------------------------------+--------------------+--------------+-----------------+--------------+
| name                           | db                 | tbl          | date_field      | retention_default |
+--------------------------------+--------------------+--------------+-----------------+--------------+
| Configuration Dynamic App Data | dynamic_app_data_% | dev_config_% | collection_time | records_config    |
+--------------------------------+--------------------+--------------+-----------------+--------------+
```

Additional details are covered here: Data Pruner is not Completing

Disable maintenance tasks

WIP: To disable a task of maintenance you need to add it to the list on table `master.system_custom_config` with field column = `disabled_data_maintenance_tasks` the column field_value should be populated with the names of the tasks that want to be disabled, the names should be separate by comas `,` this will make the process skip those tasks on the next runs e.g.:

```
UPDATE master.system_custom_config
SET field_value ='dynamic_app_cu_collector_state_maint,purge_vanished_devices'
WHERE field = 'disabled_data_maintenance_tasks'
```

you can find the task function names following this point, or also on the logs will appear like this if the process is on debug mode:

```
DEBUG maint_suite.main 89 maint_base: Running new maintenance function: ********* 'purge_vanished_devices' *********
```

Escalating to Engineering

When escalating an issue to engineering, especially when there are issues related to Daily Maintenance SIGTERMs, it is helpful to provide the following data.

Logs

When collecting logs, make sure that the Daily Maintenance process has **Debug** enabled:

- On AP2, System → Settings → Admin Processes → Wrench EM7 Core: Daily Maintenance
- Set **Debug Mode** to **Enabled** and click **Save**

After **Debug** is enabled, wait for a full cycle of Daily Maintenance to complete before collecting the debug logs. Debug logs can be found on the target appliance:

- Prior 11.2 - Collect all **silo.log.*** files in **/data/logs/** or **/var/log/em7**
- Post 11.2 - Collect all **maint_daily.log.*** files in **/var/log/sl1**

If the process crashes before it has the chance to finish, the last log entry for the process in /etc/silo.conf will likely also generate a segfault error in */var/log/kern* at the same time.

```
 2019-05-08T02:04:26.998400+00:00 ip-172-31-23-198 kernel: python2.7[8141]: segfault at 24
ip 00007f9bf22570da sp 00007fff3c4fcd80 error 6 in libpython2.7.so.1.0[7f9bf21cd000+17e000]
```

Ref: 💔EM-37006: Exception thrown in Dynamic App table maintenance task disabling auto commit CLOSED

Database Queries

After collecting the debug logs, the following data can be a helpful supplement to the logs.

- The last 10 runs of Daily Maintenance

```
SELECT start_time, end_time FROM master_logs.system_runs
WHERE sp_id = 4 and mod_id = <module_id> ORDER BY start_time DESC LIMIT 10;
```

`mod_id` should be the ID of the appliance of interest.

- Status of the Daily Maintenance expired pruner task

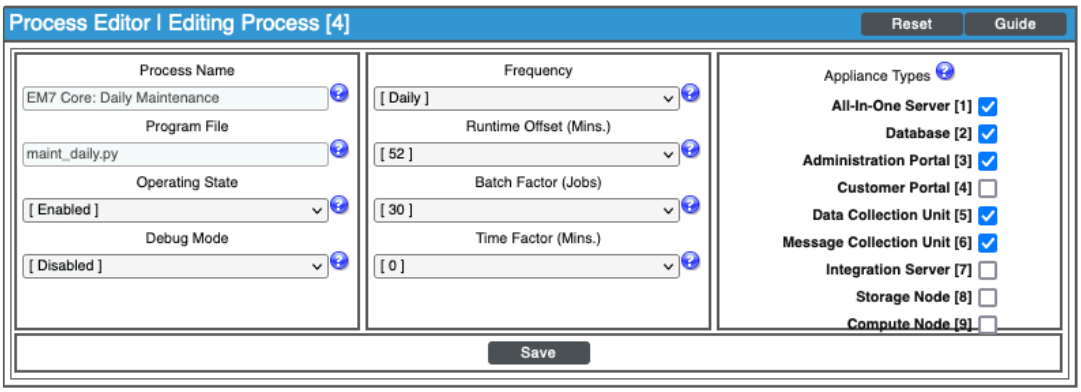
```
SELECT p_id, date_start, date_end, elapsed
FROM master_logs.pruner_log
ORDER BY date_start desc;
```

Maintenance Configuration

Take a screenshot of the Daily Maintenance configuration

- On AP2, System → Settings → Admin Processes → Wrench EM7 Core: Daily Maintenance

Example:



Post Mortem Articles - 🛠️ WIP

This section is a WIP

Daily Maintenance Pruner Limits

🐞 EM-30081: Increase daily_maint pruner minimum limit CLOSED

Daily Maintenance proper way to increase Raw Retention Data Threshold

This escalation contains a link to a KBA for changing default thresholds for raw performance data.

📋 EM-48796: Proper way to increase Raw Performance Data retention threshold to beyond 14 days to 30 days CLOSED

Daily Maintenance failing to complete because of config app pruner

There have been a few issues and resolutions related to the config app pruner.

MySQL client ran out of memory

The solutions is described on the followed confluence page:

https://confluence.sciencelogic.com/display/ENG/EM-25183%3A+Configuration+App+Data+Pruning

Resolution: 💔 EM-20781: [Defect] Overly large configuration apps can cause daily maintenance to crash due to 'MySQL client ran out of memory' CLOSED

MySQL server has gone away

The Daily Maintenance database connection was timing out because of the `maint_prune_expired_data` task taking too long to complete. Because this task uses multiprocessing, the child processes that run the individual pruners have to create their own connections to the database to prevent shared memory related issues. During this time, the parent process has an idle database connection that eventually exceeds the database's `wait_timeout` setting (default is 8 hours).

The root cause of this problem was the config app pruner taking 24+ hours to complete, largely due to the constraints imposed by product requirements. Specifically, there is a requirement to leave the last recorded value of a configuration snapshot in place even if the value's existence exceeds the system data retention setting. This results in thousands to hundreds of thousands of rows of data that persist for months/years without ever being pruned.

The problem is exacerbated by applications that have short lived indexes, resulting in large quantities (millions in EM-49808) of unique snapshots that have a value of `None` - essentially representing `NULL` data points.

This unique snapshot retention requirement adds complexity to the config app pruner which results in longer processing time.

Escalations:

EM-49808: Daily Maintenance Not Being Marked Complete CLOSED

EM-51834: Daily Maintenance, Config app pruning leaving obsolete data CLOSED

EM-54145: Performance Issue (Pruner failing to complete due to configuration dynamic app data with 0 rows deleted queries taking a very long time) CLOSED

Resolutions:

Improvements to the config app pruner became a priority as more customers began to experience performance issues.

EM-51173: Prevent idle database connection timeouts during daily maintenance execution CLOSED

- This solution solves the symptom of the config app pruner from taking too long. The Daily Maintenance process will no longer maintain an open database connection during the `maint_prune_expired_data` task, so the `MySQL server has gone away` error should no longer occur if the config app pruner takes 8+ hours to complete.

EM-38204: Config App Pruning - Bulk Deletes CLOSED

- This solution uses a Bulk API to reduce the number of individual DELETE queries that are sent to the database to prune config app data.

EM-51176: Dynamic Config App Pruner Improvement: 0 Row Queries CLOSED

- This solution eliminates 0 row deletes for the config app pruner, which was accounting for approximately 97% of individual DELETE queries in the EM-49808 escalation.

EM-51177: Dynamic Config App Pruner Improvement: Multi processing CLOSED

💔 EM-52516: Configuration Data Pruning: None CLOSED